Open-Source Malware Sandboxes Analysis: Final Draft

Course: CSEC-490 Capstone in Cybersecurity

Team:
Roberto Reade, Han Chen, Donovan Hwang, Hayden Orr

Mentor: Dr. Yin Pan
Submission Date: April 30, 2026

Introduction

Malware sandboxes allow analysts to execute malware in isolated environments and observe their real-time behaviors without risking the production system. As this approach becomes more mainstream, the number of available sandboxes grows, making it more difficult to determine which sandboxes are most effective for malware analysis.

Malware has grown over the years and shifted from aggressive and destructive to more covert and evasive methods. According to Picus Security, there has been a record of the decline of ransomware encryption and the shift to focusing on evasion and hiding (Picus Security, 2026). This shift increases the challenge of accurate malware detection and analysis. *Unveiling the Dynamic Landscape of Malware Sandboxing: A Comprehensive Review* discusses this in depth about how advancement in the technology has granted analysts with more tools and potential to create more realistic simulations while also elaborating on the ongoing challenges with evasive malware (Debas et al., 2024).

In this paper, we examine three widely-used open-source sandboxes: CAPE (CAPE, n.d.), Cuckoo3 (Cuckoo, 2020), and DRAKVUF (Singh, 2023). We deployed each platform in a controlled academic lab environment and evaluated them by executing the same Windows sample set under comparable conditions. Our goal is not to declare a universal “best” sandbox, but to compare the breadth, granularity, and practical usefulness of the forensic artifacts each platform exposes to an analyst. To do so, we used a curated corpus of fourteen Windows executables spanning benign controls and multiple malware categories, then compared the resulting reports across process, file system, registry, network, memory, and behavioral-trace dimensions. Prior work has conducted large-scale comparative analysis of malware sandboxes, such as *A Classification of Malware Sandboxes and Their Architectures* (Bistarelli, et al. 2025),

which compared the features and design of various sandboxes. Our study focuses only on open-source tools that are run in controlled academic lab settings.

This study finds that there is not a single “best” sandbox, each of three sandboxes studied excels in different areas depending on the goals. CAPE provides the most detailed and in-depth analysis output, while DRAKVUK performs strongest in terms of system-wide visibility. Cuckoo3 is the most accessible and easiest to set up but it produces less detailed reports. Those findings highlight that sandbox selection should be guided by the specific analytic goal as well as the complexity of the deployment.

Background information

The process of Malware analysis can often be categorized into static and dynamic analysis. Static analysis involves the examination of a binary's structure, source code, and metadata without actively executing the program. Static workflows often emphasize structural and metadata-oriented artifacts, which makes them comparatively safer, easier, and cheaper to scale than running unknown code directly (Singh, 2023, p. 1). However, the effectiveness of static analysis has been critically limited by the widespread adoption of runtime packers, advanced cryptographic wrappers, and sophisticated code obfuscation techniques (Komarthi, 2025; Singh 2023). Nowadays, these techniques are standard features for threat actors to scramble or conceal the true intent of the payload until it is actively unpacked in the system’s memory during execution (Singh, 2023, p. 1; Picus Security, 2026). According to the Picus Labs’ *Red Report*, it indicated a measurable decline in aggressive encryption - “Data Encrypted for Impact (T1486)” - campaigns and a broader pivot from overt disruption toward longer-lived access and evasion; the report quantifies a 38% relative decrease in encryption prevalence from 2025 to 2026 and frames this as evidence that attackers increasingly prioritize stealth and

persistence over immediate destruction (Picus Labs, 2026, pp. 6–7). Consequently, the static signature of a packed binary is greatly limited, rendering static analysis alone entirely insufficient for categorizing modern threats. Dynamic analysis addresses these shortcomings by executing suspicious payloads in controlled, isolated environments (often a sandbox) and observing their behavior in real time. By instrumenting API calls, tracing system calls, dumping memory segments, and capturing network traffic, analysts can record the functional objectives of malware rather than relying solely on static artifacts (Debas et al., 2024, p. 1-2).

The past decade's surge in malware-driven investigations has pushed dynamic analysis toward industrial-scale automation. For example, ANY.RUN's 2025 retrospect report that 6.8 million sandbox sessions were launched in 2025 and that the platform's community collected 3.8 billion indicators of compromise (IoCs) over the same period (ANY.RUN, 2026, p. 2). At volumes like these, executing every suspicious artifact through a human-led, reverse-engineering-centric workflow becomes increasingly impractical; instead, dynamic analysis is more commonly operationalized as an automated, scalable pipeline integrated directly into triage and detection workflows (Komarthi, 2025). Malware sandboxes now serve as the operational pillar of this automation. They provide a secure, isolated, and highly instrumented execution environment designed to safely detonate potentially malicious files while systematically monitoring their interactions with the underlying operating system and network infrastructure (Debas et al., 2024, pp. 1–2; ANY.RUN, 2026, p. 2). They aim to generate accurate forensic artifacts, including system call traces, file system modifications, registry alterations, network communications, and memory manipulations (Komarthi, 2025). Essentially, everything a professional would look at during manual dynamic analysis. These artifacts are then consumed

by detection engineering teams, incident responders, and machine learning classifiers to extract IoCs and map adversarial tactics, techniques, and procedures (TTPs).

Looking into sandboxes, to reason systematically about their design and evasion resistance, researchers adopt a Systematization of Knowledge (SoK) framework that organizes sandboxing along three functional dimensions (Alrawi et al., 2024). First, sandbox implementations vary between emulation, virtualization, and bare-metal execution. Emulation platforms such as QEMU enable cross-architecture analysis but suffer from high performance overhead and limited hardware transparency (Alrawi et al., 2024). Virtualization platforms using hypervisors such as Xen or KVM provide a practical balance between scalability and transparency and have become the industry standard (Alrawi et al., 2024; Singh, 2023). Conversely, bare-metal approaches offer high hardware realism but are expensive to scale and lack robust isolation boundaries (Alrawi et al., 2024). Second, monitoring paradigms differ between inside-guest and outside-guest techniques (Alrawi et al., 2024). Inside-guest monitoring injects agents or hooks into user-space (Ring 3) or kernel-space (Ring 0), providing rich semantic context but remaining highly visible to environment-aware malware capable of circumventing these monitors (Alrawi et al., 2024). Outside-guest monitoring, typically implemented via Virtual Machine Introspection (VMI) at the hypervisor layer, adopts an agentless approach that offers superior stealth because the sensors reside outside the malware's abstraction boundary (Alrawi et al., 2024; Singh, 2023). Finally, sandboxes must simulate realistic system "wear and tear," including populated user profiles, installed applications, and browsing artifacts (Alrawi et al., 2024). Without these environmental elements, evasive malware may detect the sterile analysis environment and halt execution, generating empty behavioral reports.

Within this design space, several open-source sandboxes dominate academic and operational use. Cuckoo3, maintained by CERT-EE, is the Python 3.19 successor to the deprecated Cuckoo Sandbox that was created as a Google Summer of Code project in 2010 within The Honeynet Project and was widely popular for many years (Cuckoo Sandbox, 2020; CAPE Project, n.d.). It maintains a traditional host-guest architecture, detonating malware inside isolated virtual machines and monitoring behavior through an in-guest Python agent (Cuckoo Sandbox, 2020). Its strengths include modularity, an accessible web interface, strong documentation, and integration with tools such as YARA and Volatility (Cuckoo Sandbox, 2020). However, its reliance on in-guest monitoring increases susceptibility to detection and evasion by modern malware (Singh, 2023).

Derived from the original Cuckoo framework, CAPE Sandbox offers extended capabilities with a focus on advanced payload extraction and automated dynamic unpacking (CAPE Project, n.d.). It incorporates an automated debugger that is programmable via YARA signatures, enabling analysts to dynamically extract decrypted shellcode and embedded command-and-control configurations directly from memory (CAPE Project, n.d.). While powerful for extracting actionable intelligence, its reliance on API hooking leaves it vulnerable to specific evasion techniques, prompting developers to continually add countermeasures for direct and indirect syscalls (CAPE Project, n.d.).

DRAKVUF represents a shift towards agentless analysis using Xen-based Virtual Machine Introspection (Singh, 2023). By injecting breakpoints into guest physical memory through Intel Extended Page Tables (EPT), it captures user-space and kernel-space execution without requiring in-guest software (Singh, 2023). This approach, in theory, offers exceptional stealth and resilience against evasion, making it particularly effective for analyzing fileless

malware and rootkits. However, DRAKVUF has strict hardware constraints (Intel VT-x/EPT) and requires precise operating system memory profiles, making support for newer Windows builds more difficult (Singh, 2023).

Given the open-source nature of these platforms, their internal detection logic and architecture assumptions are publicly exposed. Any realistic threat model must therefore assume that adversaries can study their mechanisms and tailor evasion techniques accordingly. We selected CAPE, Cuckoo3, and DRAKVUF due to their popularity, active development communities, and widespread adoption in Windows malware analysis workflows. Focusing on Windows-centric sandboxes enables consistent sample execution and controlled comparative evaluation within project time constraints. Although alternative frameworks such as Detux, Limon, and PokiSec are very capable tools, they fall outside the scope of this study as they target Link ecosystems.

Project Description

This project will compare three of the most popular sandboxes: CAPE Sandbox, Cuckoo 3 Sandbox, and DRAKVUF, to see the kind of dynamic-analysis visibility each platform provides under controlled Windows-based testing conditions. As a team, we examine the relative strengths and limitations of each platform in terms of the forensic artifacts it exposes, the granularity of those artifacts, and the practical usefulness of its reports for malware analysis. Instead of only a few malware specimens, we used fourteen Windows executables spanning seven categories, including benign controls, ransomware, worms, botnets, trojans, spyware or infostealers, and rootkits or bootkits. This allowed us to collect artifacts from the sandbox across variations of malware families and non-malicious software.

The technical aspects of the project will involve creating and setting up the sandbox environments, preparing the Windows victim systems, selecting the malware samples, and collecting the resulting reports produced by each sandbox. We then compared the outputs with an emphasis on analyst-relevant visibility, including process activity, file and registry changes, network artifacts, API or system-level tracing, memory outputs, screenshots, and other platform-specific features such as extracted payloads, configuration data, or interactive analysis support.

Throughout the semester, we have set up and executed the three sandboxes mentioned above, using several different setups to get them functioning. We went through the malware mentioned above, as well as two benign software programs mentioned in the next section. We then compared the three sandboxes to show the effectiveness of each in analyzing software as malicious or benign, as well as the technical details between the reports created by each.

Sample Selection

To preserve comparability across platforms (also due to limited time and resources), all selected samples were Windows executables that could be detonated against a common Windows victim environment. The sample corpus contained fourteen executables spanning seven categories. Two benign utilities were included as non-malicious controls: the standalone version of 7-Zip and Microsoft Sysinternals Process Explorer. The malicious portion of the corpus included two ransomware samples (HiveRansom and WannaCry), two worm samples (EternalRock and Raspberry Robin), two botnet samples (QakBot and TrickBot), two trojan or remote-access-tool samples (AsyncRAT and Remcos), two spyware or infostealer samples (RedLine and LummaStealer), and two rootkit or bootkit samples (BlackLotus and RustyClaw).

By using multiple malware categories, the study could observe how each sandbox handled different behavioral profiles, such as credential theft, persistence, network communications, process injection, encryption behavior, and stealth-oriented execution. Including benign administrative and utility software also provided a useful control point for observing how each sandbox contextualized non-malicious Windows activity. The full sample list and hashes are summarized in Table 1.

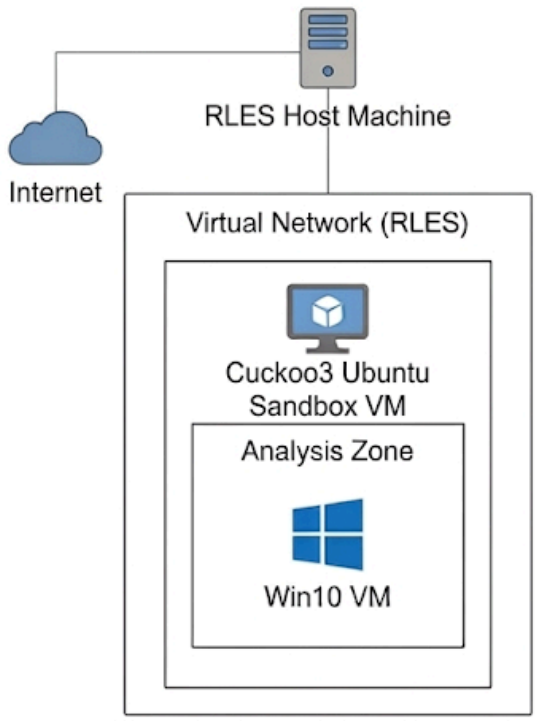
sample_id	sample_name	sha256
S01	7za	3a30d7761967ad217b90e3e3e02aabd82b13065d7401bf436adefdf75cfea72e
S02	AsyncRAT	f08b325f5322a698e14f97db29d322e9ee91ad636ac688af352d51057fc56526
S03	BlackLotus	f623dc161d4383e4d66d4d4321aa8b60300328e3d087565d65768f7d241c2a50
S04	EternalRock	cf8533849ee5e82023ad7adbdbd6543cb6db596c53048b1a0c00b3643a72db30
S05	HiveRansom	481dc99903aa270d286f559b17194b1a25deca8a64a5ec4f13a066637900221e
S06	LummaStealer	674d96c42621a719007e64e40ad451550da30d42fd508f6104d7cb65f19cba51
S07	QakBot	fece07249e0b9a9121291a7887ccc282f969de2ab924fb48b21ab92fcbf51c18
S08	RaspberryRobin	fd0a3ec3b1564210e261892d8ceb51637380d0326387605bdccaef44a25221bf
S09	Redline	ffd45c2b562d30113cb9a4823025a9a162503017e9d81fd96ddb5b98e5bb89bd
S10	Remcos	f127eced7a835fecf3453bcb307040fb4e91bfc0c63983d2a8d6c0dd72a4e5c1
S11	RustyClaw	0a02901d364dc9d70b8fcdc8a2ec120b14f3c393186f99e2e4c5317db1edc889
S12	TrickBot	1f03ab2c168de4e82e8fd52b974180683044fce88d1128fa5691abbe313a2593
S13	procexp64	ea2de63a9ef4fdda05b5b357a6b1c3ea7fea859f6e4c7beda2e80fa8cef1eca1
S14	wannacry	07c44729e2c570b37db695323249474831f5861d45318bf49ccf5d2f5c8ea1cd

[Table 1: List of Samples with SHA-256 Hashes]

Testing and Evaluation

To run and analyze each piece of malware, each sandbox had to be set up uniquely within a VM to be set up with a sandbox to run the analyzer as well as an isolated environment to run the malware. To restate our general methodology, we had three sandboxes run through VM's that each would analyze sets of malware and give evaluations based on dynamic analysis that was run. This section will go over general networking for each sandbox, as well as how each sandbox analyzes malware.

Firstly, shown below is the Cuckoo3 general network and VM setup.

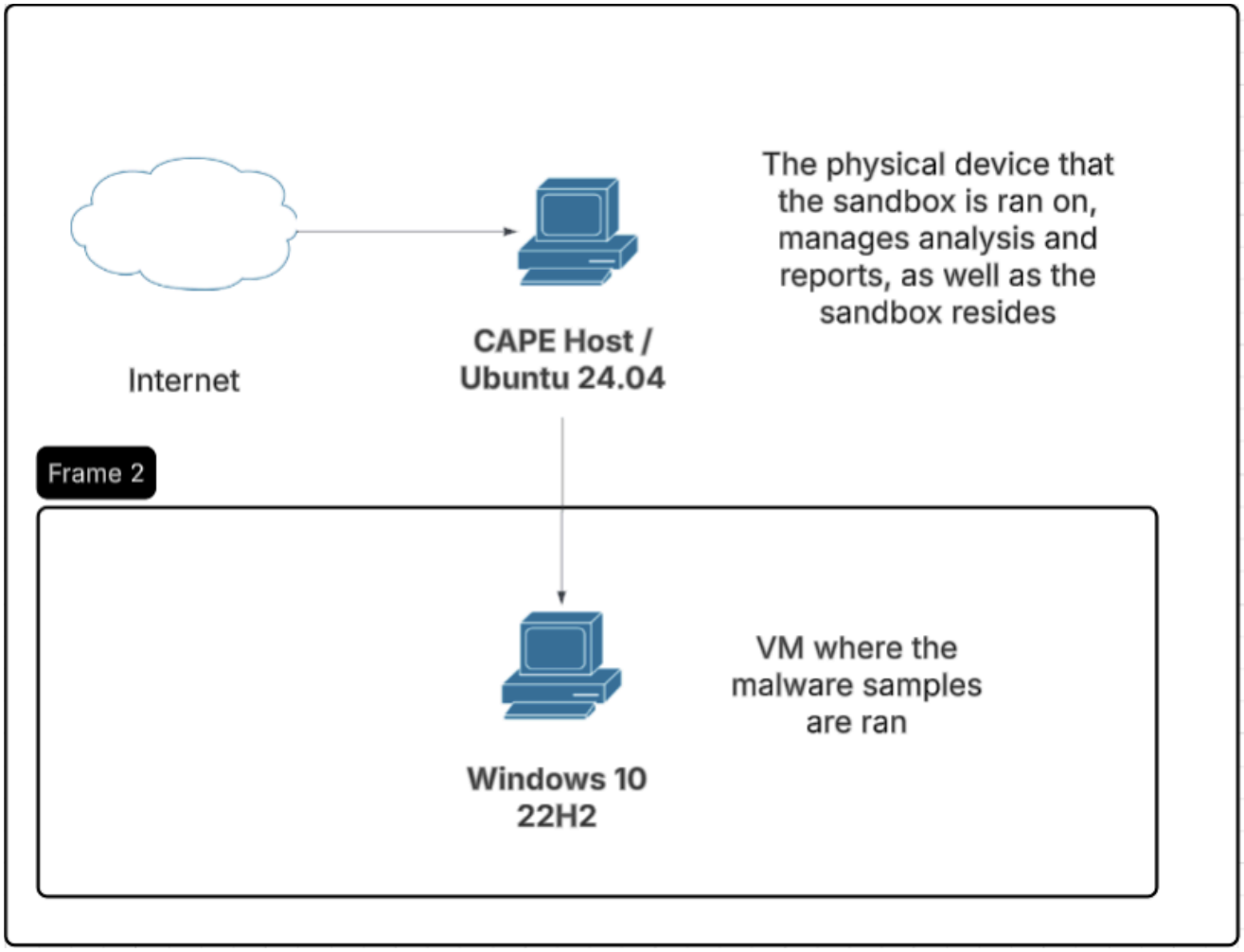


[Figure 1: Cuckoo3 Sandbox Network and Virtual Machine Architecture]

Each ring symbolizes a machine or machines that are created and used inside machines in the bigger ring. For example, the host machine in this case is the physical laptop we used, and within that is the RLES VM that we spun up within the laptop. The third ring shows a nested Windows 10 VM that we created within the RLES VM, where we directly ran the malware. Ideally, this setup would have worked for all three sandboxes, but due to technical difficulties, we made it work functionally the same using VMWare, and no nested VMs were necessary for the other sandboxes.

When the user adds a file or URL, Cuckoo 3 executes the sample in a sandbox environment to generate a report that captures everything the malware has done (cert-ee, n.d.). Prior to execution, the analysis system will perform unpacking and identification on the file using processing workers that figure out what type of file and what platform the file requires (Configuring Cuckoo, n.d.). During the execution of the sample, the file activity will be captured along with files created, deleted, or downloaded by the application, which are then reported back to Cuckoo (Vemuri, 2020). In addition, tcpdump captures network traffic sent out of the system, including connections initiated by the software (Configuring Cuckoo, n.d.). Cuckoo 3 can take memory dumps either of just the process or the entire VM (Vemuri, 2020). Post-execution, the processing workers of Cuckoo 3 compile all collected information in a report that contains details about the behavior, network activity, dropped files, and summary (Configuring Cuckoo, n.d.).

The next sandbox was CAPE, which is a Cuckoo-derived sandbox with expanded unpacking and configuration-extraction capabilities, and the setup we had for it is as follows.



[Figure 2: CAPE Sandbox Network and Virtual Machine Architecture]

Since RLES wasn't working well with CAPE, we changed the setup to be non-distributed and use VMware. This change means the physical host where we run the sandbox is the same machine that is also running the VM where we run the malware. The host laptop was an Ubuntu 24.04 machine where we held the CAPE sandbox software as well as where we had VMware running a Windows 10 box to run the malware.

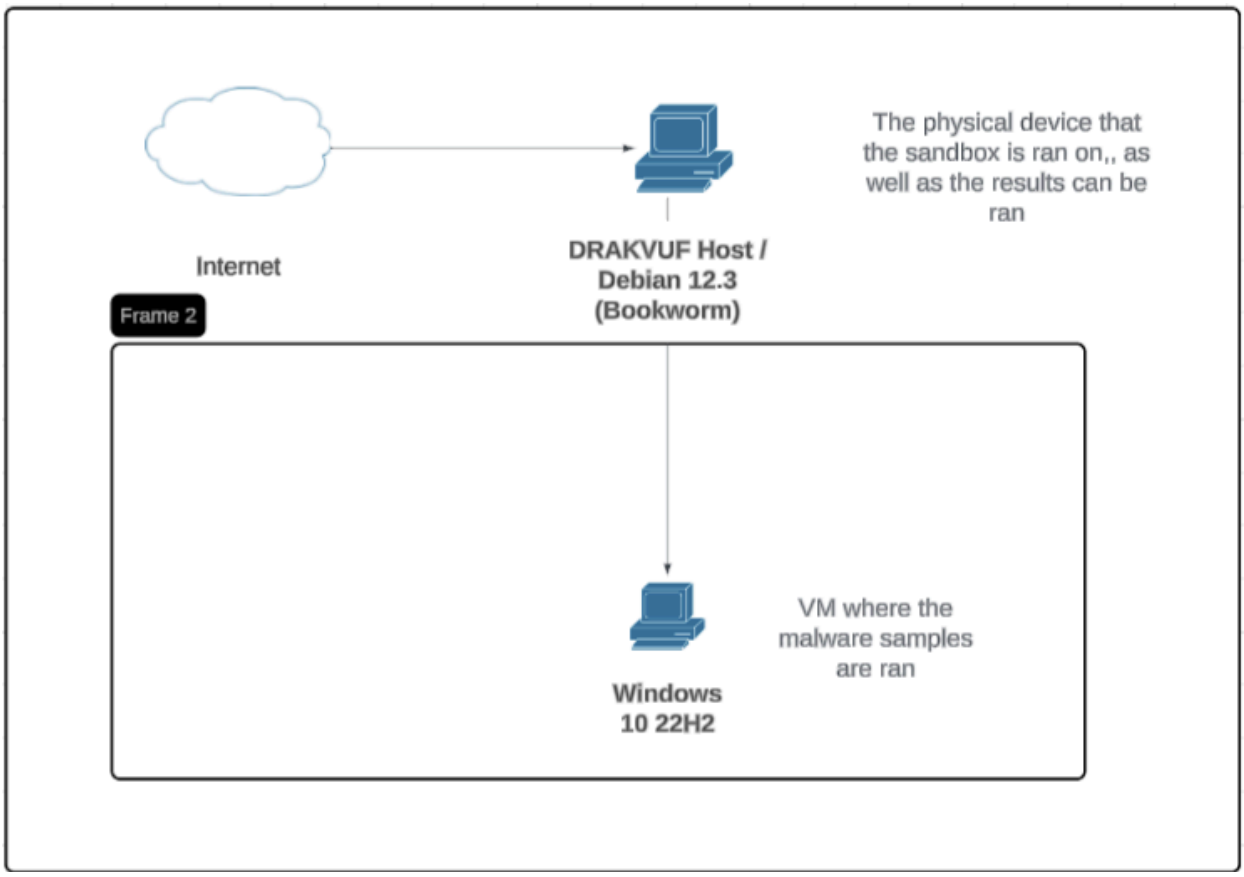
CAPE executes the suspicious file within an isolated virtual machine, monitoring all actions performed by the software. It leverages API hooking technology to observe the malware's behavior, logs any new or deleted files during execution, captures the network activity,

creates screenshots, and gathers the complete memory dump of the victim's computer (O'Reilly, n.d.). After submitting the malware, CAPE will execute the malware sample in the sandbox and generate a detailed report highlighting all actions, including the creation and deletion of files, scheduling tasks, and communication with IP addresses and domain names (Bourne, 2024). CAPE will also unpack the malware, categorize it based on YARA signatures, and extract malware configuration information (O'Reilly, n.d.). CAPE has developed a custom debugger to ensure the execution of the malware sample and circumvent evasion mechanisms employed by sophisticated malware (O'Reilly, n.d.).

Similar to the setup for CAPE, DRAKVUF was also set up to be non-distributed. The host machine in this case was a Debian 12.3 machine. On this machine, we ran the DRAKVUF client directly as well as hosted the VM where we ran the malware samples, which was a Windows 10 box hosted through VMware. One note is the slight difference between DRAKVUF analysis and the other two sandboxes in terms of how analysis is done; DRAKVUF does the malware analysis outside of the sandbox, while the other two do their analysis within the sandbox.

DRAKVUF Sandbox does not need any agent installed on the guest operating system (CERT-Polska, n.d.). It executes in-depth tracing of binaries without using any particular software within the virtualized guest OS (DRAKVUF, n.d.). DRAKVUF achieves this by executing the process on the hypervisor level. This means that it analyses malware externally and under the guest OS, giving the malware very little chance of detecting that it is being monitored. Because of this method of operation, DRAKVUF leaves a near-invisible trace, making it a perfect place to analyze any form of malware (DRAKVUF, n.d.). After submitting a suspicious file, users can then browse the results using the web portal to establish whether the file is

malicious (CERT-Polska, n.d.). For the analysis, DRAKVUF utilizes plugins that help monitor different behaviors like tracing file access, heap allocation tracking, extraction of files before deletion, and monitoring network connection activity (DRAKVUF, n.d.).



[Figure 3: DRAKVUK Sandbox Network and Virtual Machine Architecture]

Because many malware families depend on runtime libraries, user-environment artifacts, or weakened defenses in order to execute more fully, the Windows victim virtual machines were prepared to maximize observable behavior and artifact collection. In the controlled lab environment, we disabled Windows Firewall, Windows Defender, and User Account Control on the victim systems; launched PowerShell at least once to avoid first-run delays; installed multiple versions of the Visual C++ Redistributable and the .NET Framework; generated .NET native image caches; and created dummy files throughout the guest operating system.

Results

To evaluate the three sandbox platforms, we conducted a controlled comparative analysis in which the same set of malware samples was executed across CAPE, Cuckoo3, and DRAKVUF under consistent testing conditions. Rather than judging the platforms only by whether a sample successfully ran, we evaluated them based on the breadth and usefulness of the forensic artifacts they produced in their reports. For each sandbox, we reviewed whether it captured key categories of dynamic-analysis output, including process and process-tree information, network connections, DNS and HTTP activity, file and registry modifications, API-call traces, memory artifacts, screenshots, persistence mechanisms, and other advanced outputs such as extracted payloads, TLS session keys, and interactive analysis features. This capability-based evaluation allowed us to compare not only whether the sandboxes detected malicious behavior, but also how much analytical visibility each one provided to the investigator. The resulting comparison, summarized in Table 1, was then used to identify each platform's relative strengths and limitations in supporting practical malware-analysis workflows.

All three malware sandboxes share a common baseline of core analysis capabilities, including process tree visibility, network connection logging, DNS/domain capture, raw PCAP collection, and screenshots. The main differences lie in the depth and directness of artifact extraction. CAPE provides the broadest direct coverage overall, with support for file and registry activity, per-process API calls, memory dump artifacts, executed commands, extracted payloads/configurations, and TLS session keys. DRAKVUF is also strong, particularly in lower-level visibility, as it uniquely provides system-wide syscall and OS event tracing while also supporting per-process API calls, memory dump artifacts, and TLS session keys; however, several artifacts such as persistence, mutexes, and executed commands are available only indirectly through raw logs. Cuckoo3 offers the most limited direct artifact coverage of the three: while it handles the shared baseline well, it lacks direct support for HTTP/URL reporting, per-process API calls, memory dump artifacts, extracted payloads/configurations, and TLS session keys, and some categories such as file activity, registry activity, persistence, mutexes, and executed commands are only partially available through indirect or raw-log evidence. Overall, the updated comparison suggests that CAPE is the most comprehensive for direct behavioral and unpacking-oriented analysis, DRAKVUF is particularly valuable for deep low-level system tracing, and Cuckoo3 provides a more limited but still useful baseline behavioral view.

Artifact type	CAPE	Cuckoo3	DRAKVUF
Processes / process tree	✔	✔	✔
Network connections	✔	✔	✔
DNS / domains	✔	✔	✔
HTTP requests / URLs	✔	✕	✔
Raw PCAP file	✔	✔	✔
File activity	✔	△	✔
Registry activity	✔	△	✔
Per-process API calls	✔	✕	✔

System-wide syscall / OS event trace	✕	✕	✔
Memory dump artifacts	✔	✕	✔
Screenshots	✔	✔	✔
Services / startup persistence	✔	△	△
Mutexes	✔	△	△
Executed commands	✔	△	△
Extracted payloads / configs	✔	✕	✕
TLS session keys	✔	✕	✔

✔	✕	△
Yes	No	Indirect, raw-log only

[Table 2: Artifacts collected by each sandbox]

The graph below shows the severity scores as shown by the CAPE and Cuckoo3 sandboxes respectively using their built in malware scoring algorithms. The numbers are generated when each malware was run and CAPE has additional functionality in that it also shows what family it believes each malware was from or a malware with similar behaviour. DRAKVUF, unfortunately, doesn't have a scoring method at all, instead focusing solely on showing the behaviour of software run in the sandbox. In our results, Cuckoo3 tended to assign higher scores on the benign controls, while CAPE correctly labeled 7zip as clean, both platforms still labeled Process Explorer as malicious. But on the malware side, we are seeing CAPE scored some samples lower, such as BlacktLotus, and QakBot, yet it also scored others higher, including RedLine, Remcos, and TrickBot. CAPE seems to use a wider scoring spread, while Cuckoo3's scores are more tightly clustered. One possible explanation is that CAPE's broader and more layered analysis pipeline may weight behaviors differently, whereas Cuckoo3 may convert suspicious activity into severity in a more uniform way.

CAPE also has additional capability where it attempts to identify malware families based on observed behavior and matched signatures. Across our 12 malware samples, it produced family matches for only 4 samples, indicating this capability has limited usability. However,

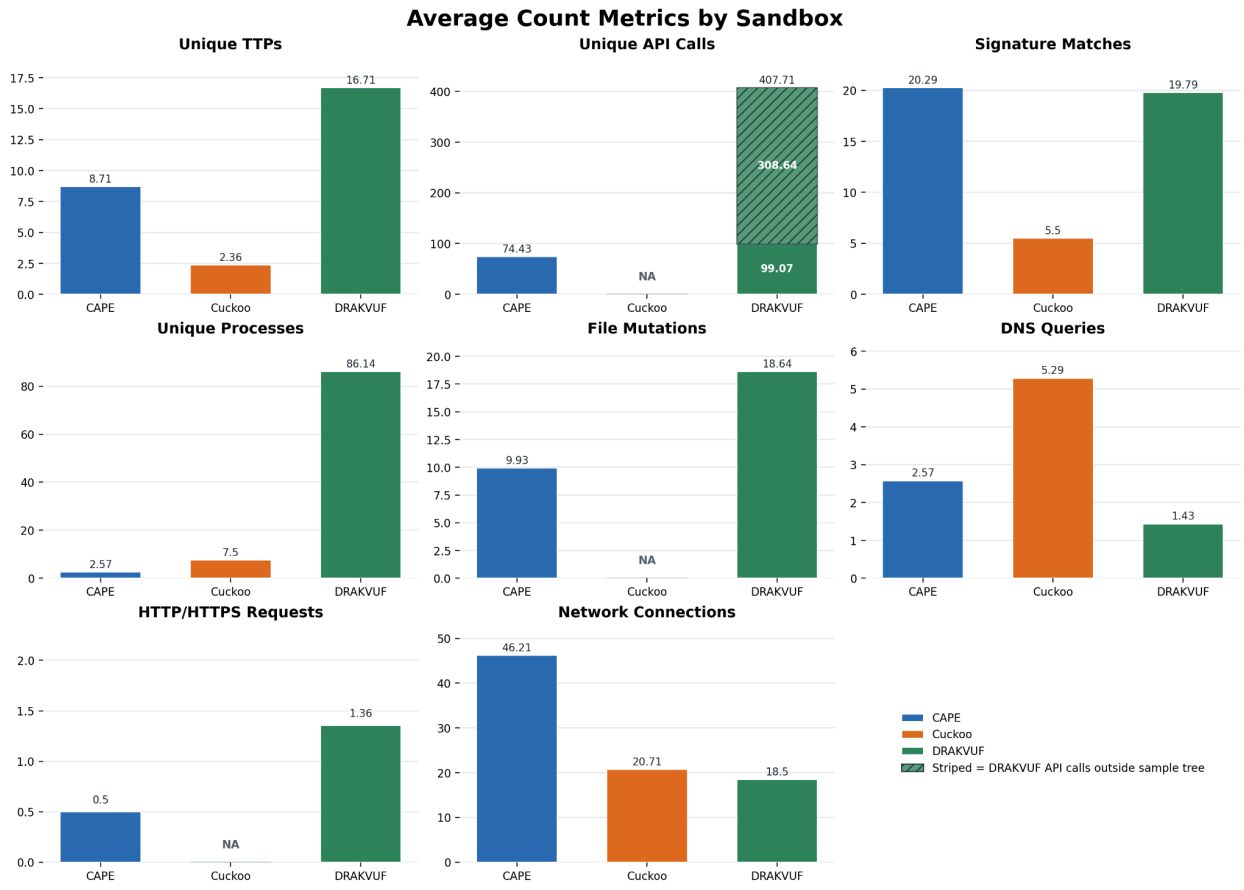
although only 2 of those were exact matches, the other two are still informative. For RedLine, CAPE matched it to Arechclient2, our research suggests they share overlapping behavioral characteristics even though Arechclient2 is typically described as having broader backdoor-style functionality than a pure infostealer like RedLine. For TrickBot, CAPE matched HiddenVNC, and we confirmed that HiddenVNC is actually used by TrickBot as one of its modules. So even when the family label is not an exact one-to-one match, it can still reveal meaningful behavioral relationships.

sample_name	severity_score/ malstatus	severity_score	detected_family
7za	0.5/Clean	4	
AsyncRAT	10/Malicious	10	AsyncRAT
BlackLotus	1.7/Clean	7	
EternalRock	9/Malicious	4	
HiveRansom	2.7/Clean	4	
LummaStealer	9/Malicious	8	
QakBot	2/Clean	9	
RaspberryRobin	8/Malicious	7	
Redline	10/Malicious	7	Arechclient2
Remcos	10/Malicious	7	Remcos
RustyClaw	8/Malicious	7	
TrickBot	10/Malicious	7	HiddenVNC
procexp64	7/Malicious	9	
wannacry	8/Malicious	8	

Color Key	CAPE	Cuckoo3

[Table 3: Sample Severity Assessment and Detection Results]

The graph below shows the average count metrics across the sandboxes, with more pronounced differences between them. Most of the graph names can be understood at a glance, but two that may be confusing are file mutations and signature matches. File mutations are actions such as creating a file, modifying a file, and deleting a file, while signature matches are behavioural detections of malware against a pre-defined rule. One key observation is that CAPE reports significantly higher numbers for network connections, largely because it captures multiple protocols, while Cuckoo3 and DRAKVUF primarily focus on TCP/UDP traffic. The DNS metric should be interpreted with caution. CAPE and Cuckoo3 both count unique DNS query name/type pairs, while DRAKVUF, in addition to DNS query name/type pair, it also includes unique port-53 connection endpoints where domain query data is absent.



[Figure 4: Average Count Metrics by Sandbox]

Challenges Encountered

For DRAKVUF, the original deployment plan was to use Xen on the RLES cloud virtual machine. This approach ultimately failed because Xen-based virtualization and Domain-0 requirements were not compatible with the cloud environment, despite we enabled vt-x/amd-v and enabling nested virtualization. To resolve this, the deployment was moved from cloud to a dedicated physical spare laptop running Debian 12.13 Bookworm, which solved the compatibility problem. While DRAKVUF's official documentation was generally the strongest and easiest to follow of the three platforms, the actual Xen deployment still required clearing several source-build and bootloader issues before the systems could successfully enter Domain-0 (Detailed solution can be found in Appendix B).

- One issue arose during the Xen 4.19.2 build process, where the firmware build step failed because Xen's default configuration referenced an outdated OVMF submodule URL. This was resolved by patching Config.mk to point to a stable GitHub revision of EDK2 and manually removing the cached firmware directory so Git would not continue using the broken source.
- A second issue involved shell environment isolation: administrative commands such as ldconfig and update-grub repeatedly appeared missing because the root shell inherited a user-style PATH that excluded /usr/sbin and /sbin. This was fixed temporarily by using absolute paths and then permanently by exporting the required system administration paths.
- A third and more critical issue involved GRUB failing to detect the Xen hypervisor even after installation. An incompletely initialized 20_linux_xen script in /etc/grub.d/ that lacked the expected path definitions for GRUB libraries and helper binaries. Removing

the problematic symlink and manually defining the required variables and tool paths in the GRUB Xen script allowed update-grub to correctly detect the hypervisor and generate boot entries.

For the CAPE sandbox, Initial attempts on Ubuntu 22.04 produced repeated problems with dependency installation, KVM and QEMU configuration, and guest virtual machine creation. These issues prevented the Windows victim VM from being created reliably and blocked malware execution. The most effective fix was to move the host platform from Ubuntu 22.04 to Ubuntu 24.04, which resolved the major compatibility problems and provided a more stable base for the deployment. Additional troubleshooting was still required after that transition. In particular, dnsmasq.service failed to start during configuration, and a virt-manager permission error prevented the Windows victim VM from being created correctly. The dnsmasq failure was resolved using the configuration steps captured in the CAPE deployment notes (Appendix C), and the virt-manager issue was fixed by moving the Windows installation ISO into /var/lib/libvirt/images/, allowing libvirt to access the file with the required permissions.

For Cuckoo3, the main challenge was that the platform's deployment process was highly order-dependent. The environment had to be installed in a strict sequence: dependencies first, then VMCloak, and finally Cuckoo3 itself. Installing components out of order caused the sandbox deployment to fail or left the environment in an unstable and partially configured state. The team also encountered a CSRF-related WebGUI issue, which was resolved by adding the required CSRF_TRUSTED_ORIGINS setting. Network capture introduced another obstacle because tcpdump initially lacked the required permissions under the default security profile. This

was resolved by applying an AppArmor bypass for tcpdump, with the full command-line steps preserved in the deployment notes (Appendix D).

Future Work/Discussion

Future work should extend this project beyond a capability-based comparison and move toward an accuracy-based evaluation of sandbox output. In the current study, the results primarily reflect which forensic artifact categories each sandbox exposed under our testing conditions, rather than whether those outputs were the most accurate or complete representation of the malware's true behavior. A stronger follow-on study could therefore use malware samples that already have professional reverse-engineering reports, validated behavioral baselines, or known ground-truth indicators, and then compare the output of each sandbox against those references. This would make it possible to evaluate not only feature presence, but also reporting fidelity, completeness, and practical analytical usefulness. Another important limitation is that the malware samples in this project were executed using largely default configurations, which may not reflect the full analytical potential of any of the three platforms.

This is especially relevant for CAPE as it is more of a continuation and extension of the original Cuckoo v1 line. The CAPE project explicitly states that it was derived from Cuckoo v1, and its codebase still retains visible Cuckoo-derived backend structure, which helps explain why the two platforms share certain foundational behaviors while still differing substantially in output depth. CAPE also supports functionality that goes well beyond a default deployment, including automated dynamic malware unpacking, malware classification based on YARA signatures, static and dynamic configuration extraction, interactive desktop support, and a programmable debugger that can be driven through YARA signatures to support custom unpacking and

configuration extractors, dynamic anti-sandbox countermeasures, and instruction traces. In addition, CAPE exposes extensive host-side configurability through files such as “cuckoo.conf”, “auxiliary.conf”, “<machinery>.conf”, “memory.conf”, “processing.conf”, “reporting.conf”, and “routing.conf”, meaning that a baseline configuration likely underrepresents the platform’s full capability envelope.

DRAKVUF is likewise worth a wider evaluation under expanded and customized configurations. Its major architectural advantage is that it is a virtualization-based, agentless black-box analysis system that performs in-depth execution tracing without requiring software to be installed inside the guest virtual machine. This hypervisor-level design gives DRAKVUF unusually strong system-wide visibility, meaning it monitors what happens throughout the full VM rather than being limited to activity observed from within individual processes or user-space API hooks. This reduces the malware’s ability to evade in-guest monitoring. At the same time, the default installation represents only one point in a much larger design space. Official DRAKVUF Sandbox documentation shows support for additional optional features such as S3-backed storage, extra DRAKVUF command-line flags for unsupported or customized plugins, and experimental Intel Processor Trace workflows that require extra plugin support and dependencies. These characteristics suggest that future work should not evaluate DRAKVUF only as a fixed baseline system, but also as an extensible instrumentation framework whose reporting depth can change significantly depending on enabled plugins and configuration choices.

Cuckoo3 should also be revisited under more thoroughly tuned conditions before stronger comparative conclusions are drawn. However, a practical challenge is that Cuckoo3 currently presents a less mature documentation experience than the other platforms. Many sections of its

own documentation are marked as incomplete or outdated, and not all configurations are yet documented. By comparison, DRAKVUF presently offers the most comprehensive, updated, and centralized official documentation of the three platforms, while CAPE's documentation, although substantial, did not fully align with our deployment experience. In our case, we had to rely in part on a third-party deployment article to successfully deploy CAPE because the official documentation path led to repeated setup errors in our environment. Accordingly, future work should evaluate not only artifact coverage and malware visibility, but also documentation completeness, deployment reproducibility, and configuration transparency, since these factors directly affect the practical usefulness of a sandbox in research and operational settings.

Another important direction for future work is operating-system diversity. The present study was intentionally constrained to Windows malware and Windows victim environments in order to preserve experimental consistency across the three sandboxes. We chose to work with Windows 10 because it was the default OS system ran for the sandboxes and it has default configurations which make it easier to run malware. Different versions, such as Windows 11 are definitely viable as OS's that can be used. However, this also limits how broadly the conclusions can be generalized. Future studies should extend the comparison to Linux and MacOS malware samples and evaluate whether the artifact coverage, visibility, and practical usefulness of each sandbox remain consistent across different operating systems.

Ultimately, a more complete continuation of this project would compare CAPE, Cuckoo3, and DRAKVUF across multiple dimensions at once: behavioral coverage, accuracy against known ground truth, sensitivity to malware family and evasion technique, and the extent to which tuning, plugins, debugger features, or routing and reporting options improve the resulting analysis.

Conclusion

This study set out to compare three open-source malware sandboxes, CAPE, Cuckoo3, and DRAKVUF, by executing the same set of fourteen Windows executables across all three platforms under controlled lab conditions and evaluating the forensic artifacts each one produced. Rather than declaring a single best platform, the results show that each sandbox occupies a distinct position in the open-source analysis landscape. CAPE demonstrated the deepest output for payload-oriented analysis, providing extracted configurations, executed commands, per-process API call traces, and YARA-based classification that the other two platforms did not offer in their default configurations. Cuckoo3 provided a more accessible and modular experience but produced comparatively shallower reports, particularly in areas like HTTP visibility and API tracing. DRAKVUF stood out for its agentless, hypervisor-level monitoring approach, which yielded strong system-wide visibility through syscall tracing and TLS key capture while remaining more difficult for malware to detect than in-guest alternatives.

At the same time, there are some small things to keep in mind when translating to a real-world application. All three platforms were tested under largely default configurations. In that sense, the comparison reflects a base functionality rather than a ceiling-level evaluation of any platform. The deployment experience also surfaced practical considerations that do not show up in a feature comparison table but matter a great deal in real-world use. DRAKVUF required the most involved initial setup, including Xen hardware compatibility constraints and several source-build issues that had to be resolved manually. CAPE's deployment was complicated by environment-specific dependency conflicts that ultimately required migrating from Ubuntu 22.04 to Ubuntu 24.04, and the official documentation did not fully reflect the steps needed in our

environment. Cuckoo3 was the most sensitive to installation order, with out-of-sequence steps leaving the environment in unstable states.

Taken together, the findings suggest that sandbox selection should be driven by specific goals rather than by a single performance metric. Analysts prioritizing deep payload extraction and malware family identification may find CAPE's extended capabilities more valuable, while those working in environments that require stealth against evasion-aware malware may benefit more from DRAKVUF's agentless design. Cuckoo3 remains a reasonable starting point for teams newer to dynamic analysis, given its accessible interface and established documentation, though its output depth currently lags behind the other two platforms. Future work that tests these platforms against known behavioral baselines, across non-Windows operating systems, and under more thoroughly tuned configurations would provide a more complete picture of where each sandbox's analytical ceiling actually sits.

References

ANY.RUN. (2026, January 20). *Malware trends overview report: 2025*. Medium. https://medium.com/@anyrun/malware-trends-overview-report-2025-8aba11baacd3

Alrawi, O., Wong, M. Y., Avgetidis, A., Valakuzhy, K., Adjibi, B. V., Karakatsanis, K., Ahamad, M., Blough, D., Monrose, F., & Antonakakis, M. (2024). *SoK: An essential guide for using malware sandboxes in security applications: Challenges, pitfalls, and lessons learned*. Cornell University Library, arXiv.org.

Bistarelli, S., Burberi, E., & Santini, F. (2025). A classification of malware sandboxes and their architectures. In G. Costa, R. Montanari, M. Carminati, & G. Sciarretta (Eds.), Proceedings of the Joint National Conference on Cybersecurity (ITASEC & SERICS 2025) (CEUR Workshop Proceedings, Vol. 3962). CEUR-WS.org. https://ceur-ws.org/Vol-3962/paper1.pdf

Bourne, C. (2024, June 10). *Automating Malware Analysis using CAPE Sandbox - Endure Secure*. Endure Secure. https://endsec.au/blog/building-an-automated-malware-sandbox-using-cape/

CAPE Project. (n.d.). What is CAPE? In *CAPEv2 documentation*. Read the Docs. https://capev2.readthedocs.io/en/latest/introduction/what.html

cert-ee. (n.d.). *cuckoo3/README.md at main · cert-ee/cuckoo3*. GitHub. Retrieved April 10, 2026, from https://github.com/cert-ee/cuckoo3/blob/main/README.md

CERT-Polska. (n.d.). *GitHub - CERT-Polska/drakvuf-sandbox: DRAKVUF Sandbox - automated hypervisor-level malware analysis system*. GitHub. Retrieved April 10, 2026, from https://github.com/CERT-Polska/drakvuf-sandbox

Configuring Cuckoo. (n.d.). Cert.ee. Retrieved April 10, 2026, from https://cuckoo-hatch.cert.ee/static/docs/configuring/cuckoo/

Cuckoo sandbox and process monitor (procmon) performance evaluation in large-scale malware detection and analysis. (n.d.). British Journal of Computer Networking and Information Technology. https://doi.org/10.52589/bjcnitfcedoomy

Cuckoo Sandbox. (2020, June 27). *Cuckoo Sandbox book* (Release 2.0.7). https://cuckoo.readthedocs.io/en/latest/

Debas, E., Alhumam, N., & Riad, K. (2024). Unveiling the Dynamic Landscape of Malware Sandboxing: A Comprehensive Review. *International Journal of Advanced Computer Science and Applications (IJACSA)*, *15*(3). http://dx.doi.org/10.14569/IJACSA.2024.01503137

DRAKVUF. (n.d.). *DRAKVUF Black-box Binary Analysis System*. Drakvuf.com. Retrieved April 10, 2026, from https://drakvuf.com/

Gkritsis, E., Patsakis, C., & Stergiopoulos, G. (2025). Exploiting Data Structures for Bypassing and Crashing Anti-Malware Solutions via Telemetry Complexity Attacks. 10.48550/arXiv.2511.04472.

Komarthi, J. (2025). Evaluation of Malware Analysis Tools: Lastline, ReversingLabs, and Sonic Sandbox Engine. *International Journal on Science and Technology*, *16*, 10.71097/IJSAT.v16.i4.10088.

O'Reilly, K. (n.d.). *kevoreilly/CAPEv2*. GitHub. Retrieved April 10, 2026, from https://github.com/kevoreilly/CAPEv2

Persistence Market Research. (2026, February 5). *Malware analysis Market growth accelerates as cyber threats become more sophisticated*. openPR.com.

https://www.openpr.com/news/4378270/malware-analysis-market-growth-accelerates-as-cyber-threats

Picus Security. (2026). *The Red Report™ 2026: The top 10 most prevalent MITRE ATT&CK® techniques* [Annual threat intelligence report]. https://www.picussecurity.com/red-report

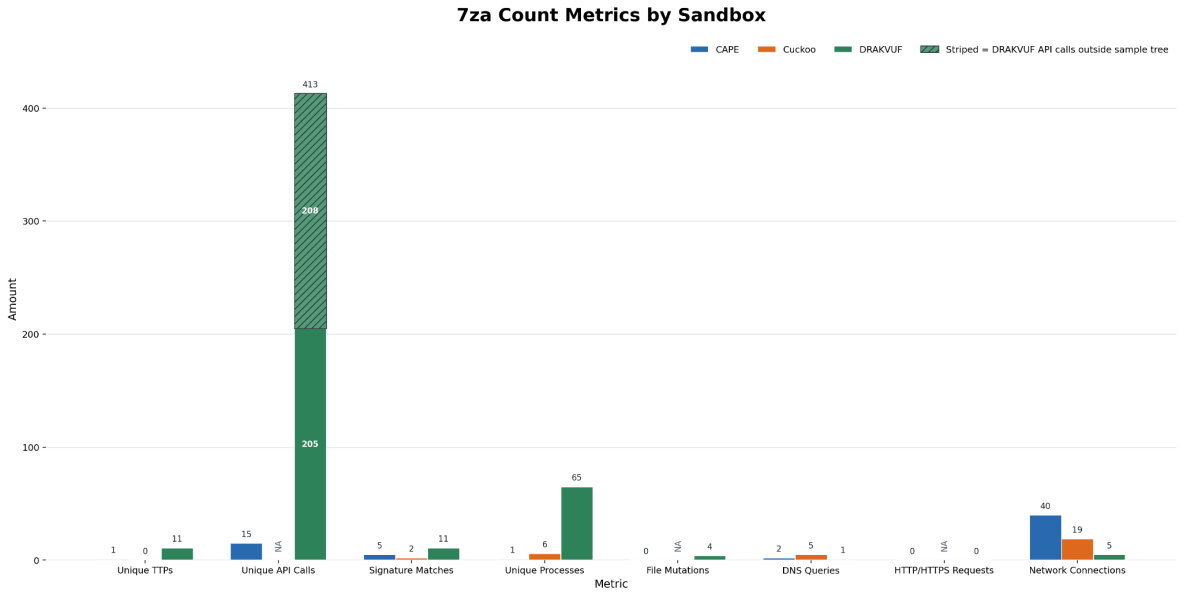
Singh, S. (2023). DRAKVUF Malware Sandbox. *Wellington Faculty of Engineering Symposium*. Retrieved from https://ojs.victoria.ac.nz/wfes/article/view/8370

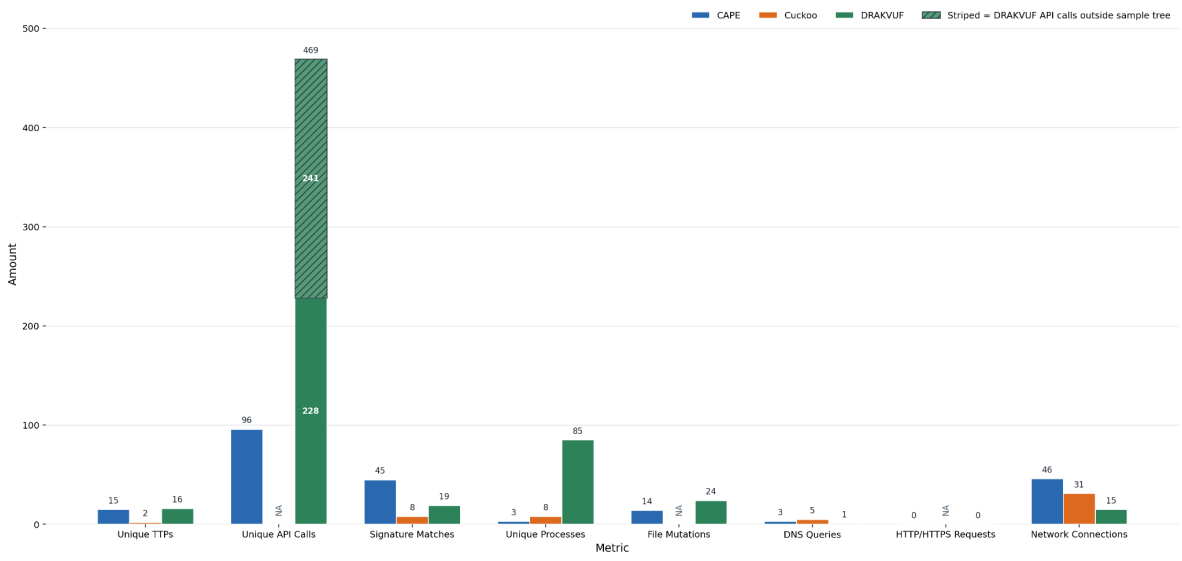
Vemuri, M. (2020, August 24). *Malware Analysis using Cuckoo Sandbox*. Medium. https://medium.com/@easypeazyo14/malware-analysis-using-cuckoo-sandbox-756616e6e85e

Appendix

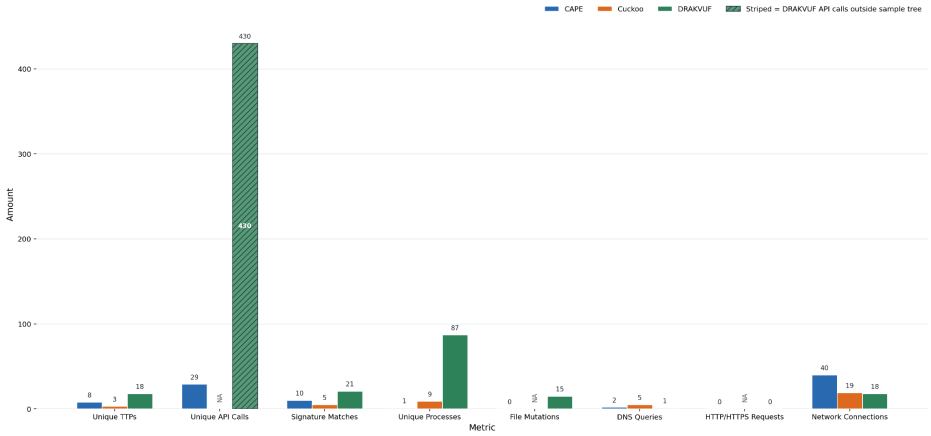
A.1 - Additional Result Data Sheets:
More detailed data sheets that supports the graphs can be found at: Metrics

A.2 - Individual graphs of the samples' metric group by the metric name:

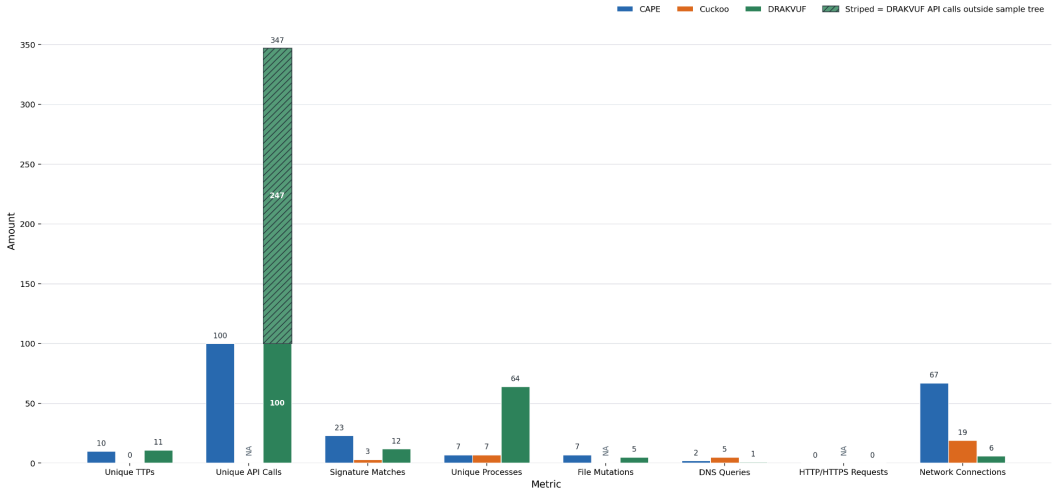




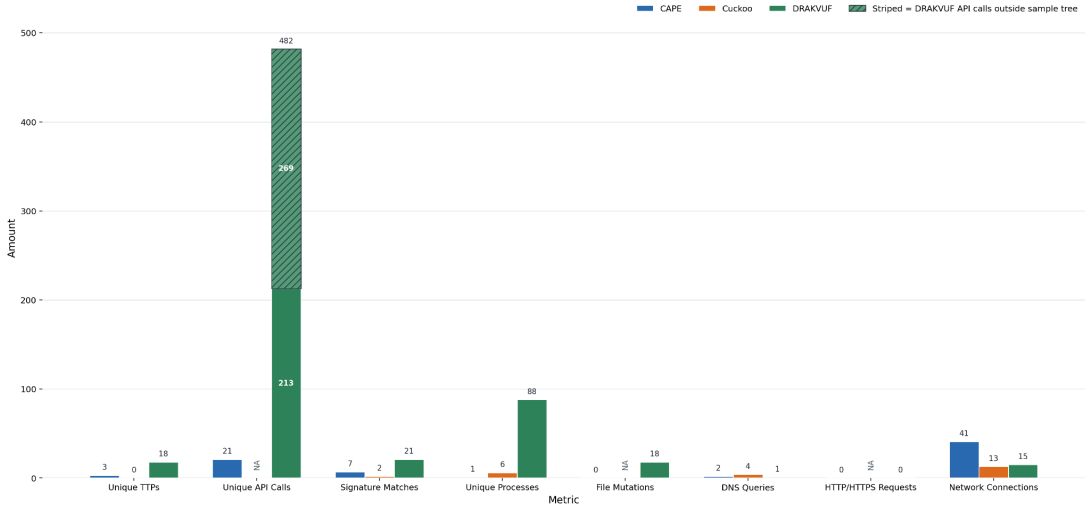
BlackLotus Count Metrics by Sandbox



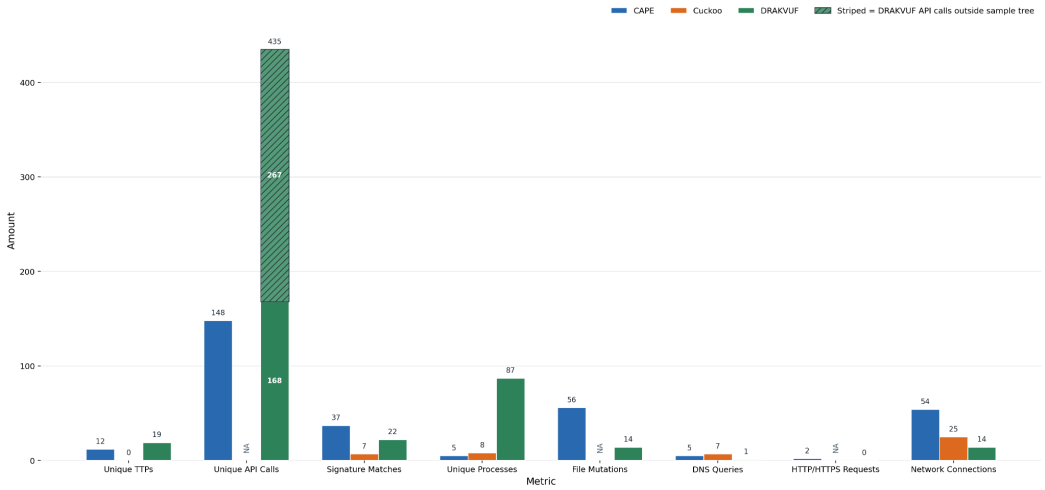
EternalRock Count Metrics by Sandbox



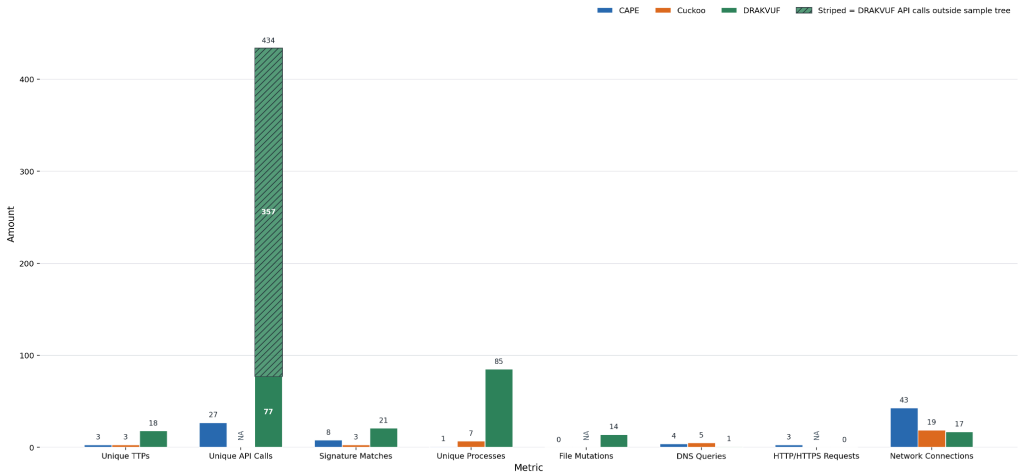
HiveRansom Count Metrics by Sandbox

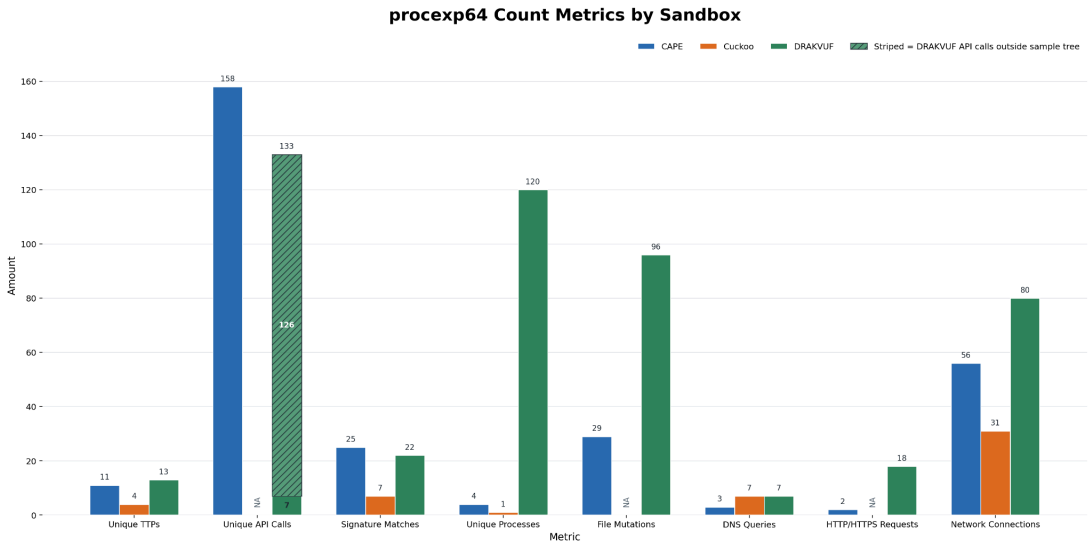


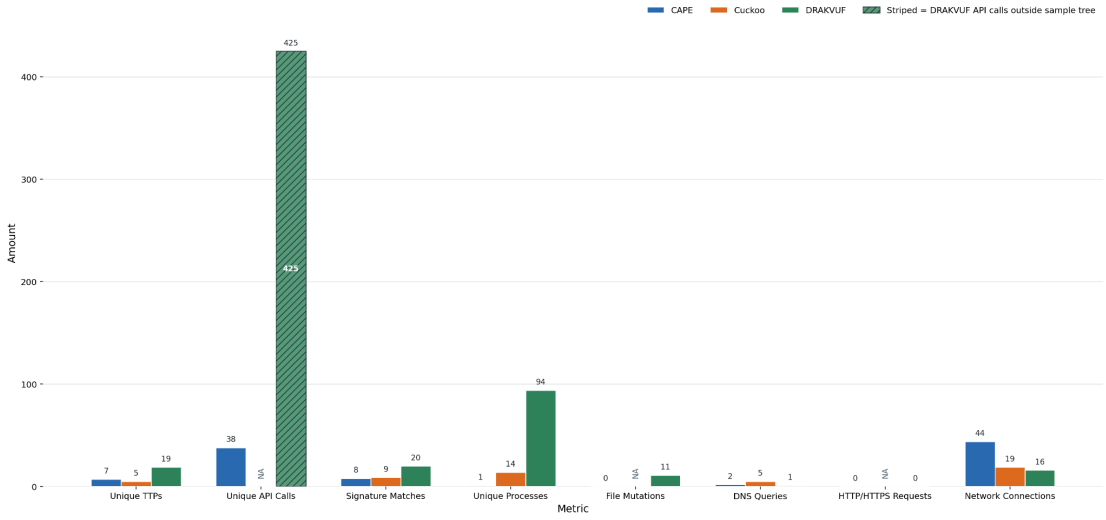
LummaStealer Count Metrics by Sandbox

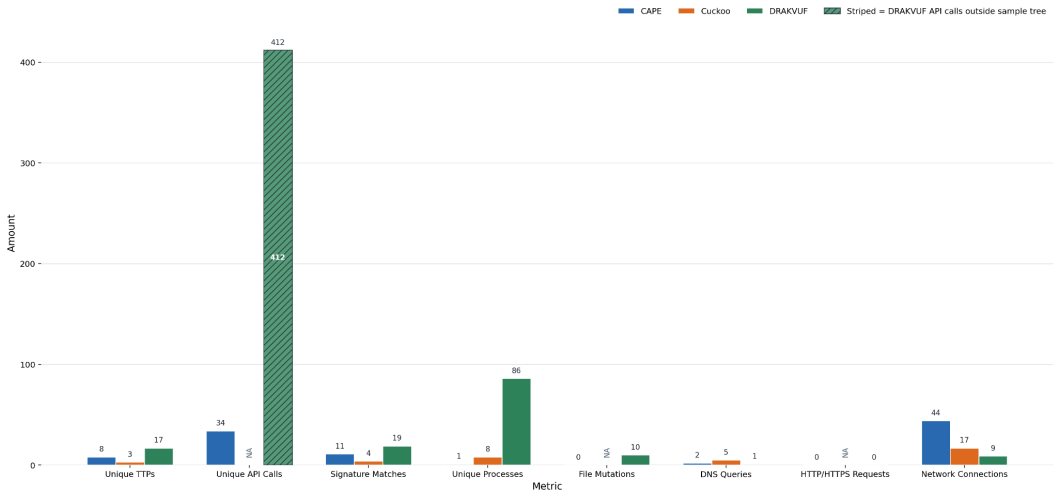


RustyClaw Count Metrics by Sandbox

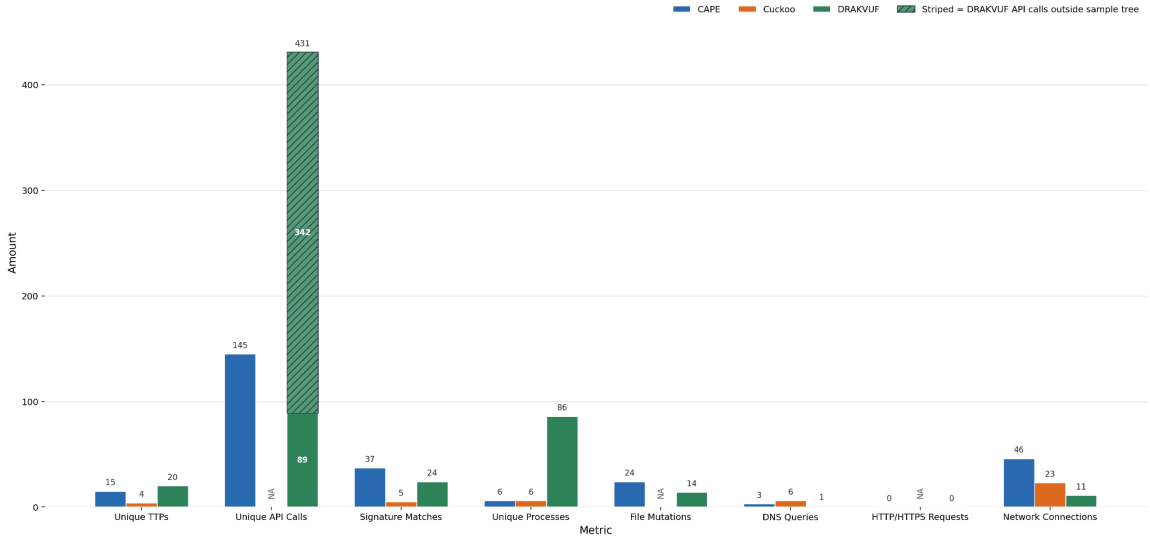



procexp64 Count Metrics by Sandbox
CAPE
Cuckoo
DRAKVUF
Striped = DRAKVUF API calls outside sample tree
Amount
Metric
Unique TTPs
Unique API Calls
Signature Matches
Unique Processes
File Mutations
DNS Queries
HTTP/HTTPS Requests
Network Connections
11
4
13
158
NA
133
126
7
25
7
22
4
1
120
29
NA
96
3
7
7
2
NA
18
56
31
80

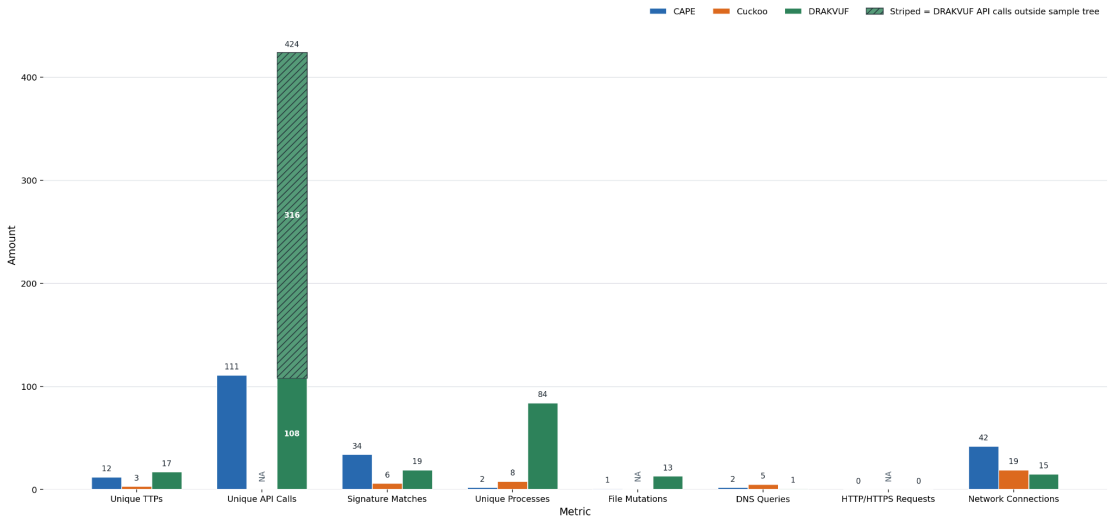

QakBot Count Metrics by Sandbox
CAPE
Cuckoo
DRAKVUF
Striped = DRAKVUF API calls outside sample tree
Amount
Metric
Unique TTPs
Unique API Calls
Signature Matches
Unique Processes
File Mutations
DNS Queries
HTTP/HTTPS Requests
Network Connections
7
5
19
38
NA
425
425
8
9
20
1
14
94
0
NA
11
2
5
1
0
NA
0
44
19
16


RaspberryRobin Count Metrics by Sandbox
CAPE
Cuckoo
DRAKVUF
Striped = DRAKVUF API calls outside sample tree
Amount
Metric
Unique TTPs
Unique API Calls
Signature Matches
Unique Processes
File Mutations
DNS Queries
HTTP/HTTPS Requests
Network Connections
8
3
17
34
NA
412
412
11
4
19
1
8
86
0
NA
10
2
5
1
0
NA
0
44
17
9

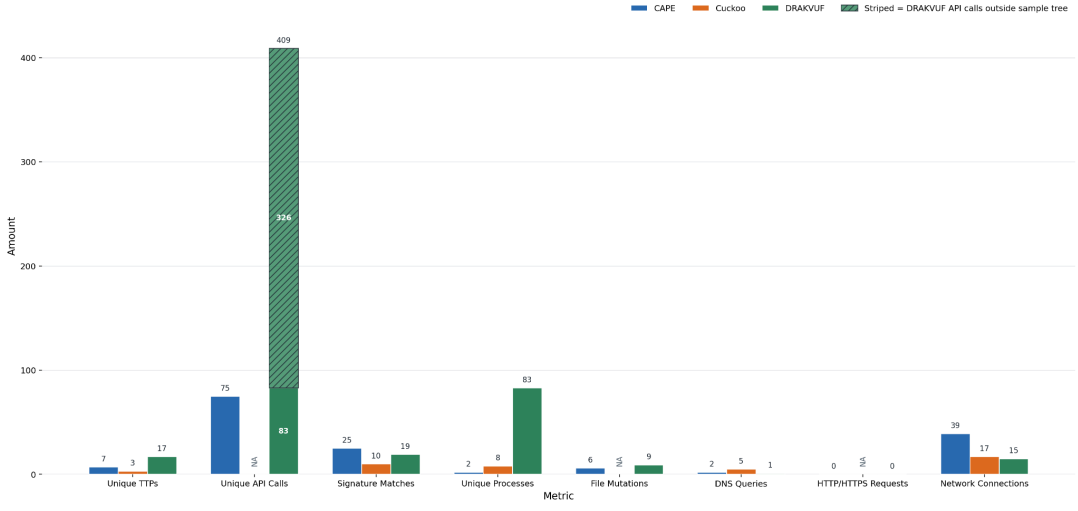
Redline Count Metrics by Sandbox



Remcos Count Metrics by Sandbox



TrickBot Count Metrics by Sandbox



Appendix B.1 - DRAKVUF / Xen Deployment Fixes

B.1.1 - OVMF / Tianocore submodule fix

```
Shell
# Remove the cached broken firmware checkout
rm -rf tools/firmware/ovmf-dir-remote

# modify Config.mk
- OVMF_UPSTREAM_URL ?= https://xenbits.xen.org/git-http/ovmf.git -
OVMF_UPSTREAM_REVISION ?= ba91d0292e593df8528b66f99c1b0b14fadc8e16
+ OVMF_UPSTREAM_URL ?= https://github.com/tianocore/edk2.git
+ OVMF_UPSTREAM_REVISION ?= 4dfdca63a93497203f197ec98ba20e2327e4afe4
```

B.1.2 Root shell PATH correction

```
Shell
export PATH=$PATH:/usr/sbin:/sbin:/usr/local/sbin
```

B.1.3 /etc/grub.d/20_linux_xen variable initialization fix

```
Shell
prefix="/usr"
exec_prefix="/usr"
datarootdir="/usr/share"

pkgdatadir="${datarootdir}/grub"
sbindir="/usr/sbin"
bindir="/usr/bin"

. "$pkgdatadir/grub-mkconfig_lib"
grub_probe="${sbindir}/grub-probe"
grub_file="${bindir}/grub-file"
grub_editenv="${bindir}/grub-editenv"

# after fixing the script then run:
update-grub
```

Appendix C - CAPEv2 deployment note

- Cape Deployment Note

Appendix D - Cuckoo3 deployment note

- Cuckoo3 Deployment Notes